

Progettazione e Sviluppo di un Video Editor Desktop in Scala 3

**Relazione di Progetto – Corso di Paradigmi di Programmazione e
Sviluppo**

Federico Bonetti
`federico.bonetti2@studio.unibo.it`

2 ottobre 2026

Indice

1	Introduzione	4
1.1	Dominio e Obiettivo del Progetto	4
1.2	Panoramica Tecnologica	5
2	Processo di Sviluppo	6
3	Analisi dei Requisiti	7
3.1	Requisiti Funzionali	7
3.2	Requisiti Non Funzionali e Vincoli di Dominio	8
4	Modello del Dominio	8
4.1	Entità Fondamentali e Algebraic Data Types	8
4.2	Invarianti di Dominio e Regole di Coerenza Temporale	10
5	Architettura e Design del Sistema	10
5.1	Struttura Modulare e Layer Architettureali	11
5.2	Disaccoppiamento del Rendering: Il Ruolo di <code>EffectCalculator</code>	13
5.3	Reattività dell'Interfaccia e Monitoraggio dell'Esportazione . .	13
6	Implementazione	13
6.1	Dettagli Implementativi: Motore di Editing ed Elaborazione Video	13
6.1.1	Manipolazione della Timeline (<code>TimelineEngine</code>)	14
6.1.2	Costruzione Dichiarativa della Pipeline FFmpeg (<code>ExportCommandBuilder</code>)	14
6.1.3	Esecuzione Asincrona e Monitoraggio dell'Avanzamen- to (<code>VideoExporter</code>)	16
6.1.4	Logica Deterministica degli Effetti Visivi tramite Ty- peclass	17
6.2	Dettagli Implementativi: Interfaccia Utente e Coordinamento Reattivo	18
6.2.1	Composizione Visuale della Timeline e Manipolazione Diretta	18
6.2.2	Sincronizzazione della Preview e Rendering degli Effetti	19
6.2.3	Mixer Audio Multitraccia e Gestione dello Stato Con- corrente	20
6.2.4	Ispezione Contestuale dei Parametri (<code>EffectInspectorPane</code>)	20
6.3	Dettagli Implementativi: Il controller	21
6.3.1	Gestione Atomica dello Stato e Cronologia (<i>Single Sour- ce of Truth</i>)	21
6.3.2	Playback Loop e Sincronizzazione Temporale Differen- ziale	22
6.3.3	Elaborazione Asincrona dell'Esportazione con Future . .	22

7	Strategia di Testing e Validazione	23
7.1	Verifica degli Invarianti di Manipolazione	23
7.1.1	Preservazione dell’Immutabilità e Robustezza agli Indi- ci Invalidi	23
7.1.2	Taglio Deterministico e Sincronizzazione Multitraccia . .	24
7.1.3	Inserimento con Ripartizione e Shift Temporale	24
7.1.4	Compattazione Magnetica (<i>Snapping</i>) e Gestione del Disordine	25
7.1.5	Polimorfismo delle Clip Visive (<i>ImageClip</i>)	26
7.1.6	Ulteriori test	26
8	Conclusioni e Sviluppi Futuri	26
8.1	Sintesi del Lavoro Svolto e Raggiungimento degli Obiettivi . . .	27
8.2	Autovalutazione e Criticità	27
8.3	Sviluppi Futuri	28
8.3.1	Obiettivi Prioritari per il Modulo Advanced Software Engineering and Development (Integrazione con Agenti IA)	28
8.3.2	Evoluzioni a Lungo Termine ed Efficienza di Sistema . .	29

1 Introduzione

1.1 Dominio e Obiettivo del Progetto

Il presente progetto si colloca all'interno di un'iniziativa di sviluppo più ampia, concepita in modo modulare per combinare i paradigmi della programmazione funzionale applicati al settore multimediale con tecniche di intelligenza artificiale, sfruttando le astrazioni e gli strumenti messi a disposizione dal motore applicativo. Come concordato nell'ambito del piano formativo, l'architettura complessiva è suddivisa in due componenti autonome e sinergiche, affrontate e valutate in momenti distinti:

- **Il Core Multimediale e Motore NLE (oggetto del presente lavoro):** la progettazione e l'implementazione in Scala 3 del video editor desktop, comprendente la gestione di una timeline multitraccia, un modello di dominio immutabile, il calcolo e la gestione degli effetti visivi e la pipeline di esportazione finale tramite processi nativi FFmpeg.
- **Il Modulo di Automazione Intelligente (estensione successiva):** un sottosistema ad agenti e modelli generativi pensato per l'editing assistito e la generazione di montaggi da prompt testuali, progettato per integrarsi direttamente sopra le funzionalità offerte da questo motore.

Questo elaborato si concentra quindi sulla realizzazione del motore principale e dell'architettura applicativa.

L'elemento cardine attorno a cui ruota l'intero sistema è la **Timeline**. Concettualmente, la timeline rappresenta l'asse temporale globale su cui si sviluppa il montaggio, organizzata verticalmente su molteplici **tracce** concorrenti e parallele lungo cui disporre le risorse multimediali.

All'interno del dominio applicativo si distinguono due categorie concettuali fondamentali:

- **Elementi Visivi e Tracce Video:** risorse caratterizzate da una componente visiva e spaziale. In questa categoria rientrano sia i filmati video, sia le **immagini statiche**, trattate a livello logico come risorse a fotogramma invariante con una durata temporale configurabile. A ciascun elemento visivo è possibile associare filtri grafici o trasformazioni dinamiche nel tempo (come regolazioni di luminosità, dissolvenze, zoom o vibrazioni).
- **Elementi Sonori e Tracce Audio:** flussi puramente acustici (musica, parlato ed effetti sonori). Ciascun elemento sonoro include il controllo del proprio livello di volume.

I normali file video contengono solitamente sia la traccia visiva sia quella audio racchiuse nello stesso file sorgente. Nel flusso di lavoro, al momento dell'importazione il file viene scisso automaticamente nelle sue due entità logiche disaccoppiate. Questa separazione assicura piena libertà operativa

nel montaggio non lineare, consentendo di tagliare, spostare o eliminare la parte video indipendentemente da quella audio (e viceversa).

L'utente può eseguire le principali operazioni di editing:

- inserimento, riposizionamento ed eliminazione di clip;
- traslazione temporale lungo l'asse della traccia;
- *trimming* (rimodulazione dei punti di attacco e stacco rispetto alla durata originale della sorgente);
- divisione e taglio (*split*) di una clip in due parti contigue;
- regolazione del guadagno audio e mixaggio multitraccia;
- assegnazione e configurazione di effetti grafici alle risorse visive.

1.2 Panoramica Tecnologica

Lo stack tecnologico adottato per la realizzazione del progetto:

- **Scala 3:**
Scelto come linguaggio di riferimento per l'intero sistema. Il motore applicativo sfrutta le funzionalità del linguaggio per definire un modello di dominio espressivo.
- **ScalaFX:**
Impiegato per la costruzione della GUI desktop. L'interfaccia si avvale delle proprietà reattive ed osservabili (`ObjectProperty`) per mantenere sincronizzata la visualizzazione dello stato applicativo con le interazioni dell'utente.
- **FFmpeg (Elaborazione Multimediale ed Export):**
Viene usato per gestire la decodifica dei file sorgente, l'applicazione degli effetti e il rendering finale del video. L'integrazione sfrutta l'interfaccia di processo di Scala (`scala.sys.process`) per lanciare la Command Line Interface di FFmpeg come processo esterno. In particolare, il motore si occupa di:
 - generare la catena di filtri audio e video (`-filter_complex`) necessaria per combinare le tracce, gestire le sovrapposizioni e applicare gli effetti;
 - delegare il rendering a un processo separato, evitando che l'interfaccia grafica si blocchi del tutto durante l'elaborazione;
 - leggere l'output del processo (`stderr`) per calcolare la percentuale di completamento e aggiornare in tempo reale la barra di avanzamento nella finestra di export.

2 Processo di Sviluppo

Lo sviluppo del sistema ha seguito un approccio **iterativo e incrementale** basato su *vertical slices*, affiancato da pratiche di **Test-Driven Development (TDD)** per i componenti critici del dominio. Trattandosi di un'applicazione multimediale in cui la resa visiva e la manipolazione dei file sono centrali, sviluppare l'applicazione per porzioni verticali ha consentito di verificare fin da subito l'effetto delle modifiche sia a livello di modello sia a livello visivo e di rendering.

L'avanzamento del lavoro si è articolato nei seguenti passaggi per ciascuna funzionalità:

1. **Modellazione del Dominio e TDD (Red-Green-Refactor):** La logica pura del modello (invarianti temporali, calcolo degli intervalli, validazione delle tracce) è stata sviluppata seguendo il ciclo *Red-Green-Refactor*.
2. **Integrazione con l'Engine (FFmpeg):** Implementazione della logica di conversione tra le strutture del dominio e i comandi per la CLI di FFmpeg, gestendo l'invocazione dei processi e l'acquisizione dei flussi di output;
3. **Interfaccia Utente (ScalaFX):** Esposizione dei controlli a video e sincronizzazione reattiva dello stato tramite le proprietà della GUI (`ObjectProperty`);
4. **Verifica e Consolidamento:** Esecuzione dei test di regressione su tutta la suite, collaudo manuale dei casi limite (es. formati non supportati o percorsi non validi) e commit delle modifiche verificate.

Per garantire la riproducibilità della build e prevenire regressioni durante lo sviluppo, è stata integrata una pipeline di **Continuous Integration** tramite *GitHub Actions*.

Il workflow viene eseguito automaticamente su macchine virtuali isolate a ogni `git push`, svolgendo le seguenti operazioni:

- **Configurazione dell'ambiente:** allestimento automatico del container con Java Development Kit (JDK) e Sbt;
- **Compilazione del progetto:** verifica formale della correttezza del codice e del type checking tramite il compilatore Scala 3;
- **Esecuzione automatica dei test:** lancio del comando `sbt test` per convalidare gli invarianti del dominio, il calcolo delle coordinate temporali e la coerenza dei flussi multimediali;
- **Esito della build:** segnalazione visiva immediata dello stato della build (successo o fallimento), garantendo che la versione remota rimanga costantemente verificata e compilabile.

3 Analisi dei Requisiti

3.1 Requisiti Funzionali

- **RF1 (Importazione Media Multiformato):** Il sistema deve consentire l'acquisizione di flussi video, tracce audio e immagini fisse nei formati multimediali standard (es. MP4, MKV, MP3, WAV, PNG, JPG), estraendone la durata temporale sorgente (*source length*) per l'integrazione nel progetto.
- **RF2 (Composizione Multitraccia su Timeline):** L'utente deve poter inserire, ordinare e traslare temporalmente clip visive e sonore lungo tracce parallele indipendenti (VideoTrack, AudioTrack).
- **RF3 (Manipolazione Clip - Trimming, Split e Rimozione):** L'utente deve poter ridefinire l'istante iniziale e la durata di ogni clip (*trimming*), suddividerla in due spezzoni distinti nel punto del cursore (*split*) o rimuoverla dalla traccia.
- **RF4 (Applicazione Effetti Visivi Parametrici):** Il sistema deve consentire l'applicazione e la configurazione di trasformazioni dinamiche (Zoom progressivo, oscillazione/Shake), regolazioni cromatiche (Luminosità, Scala di grigi, Seppia, Inversione) e dissolvenze (Fade In) su singole clip visive.
- **RF5 (Regolazione Parametri Audio):** L'utente deve poter calibrare il guadagno di volume delle clip sonore collocate sulle tracce audio.
- **RF6 (Navigazione Temporale e Playback):** L'interfaccia deve fornire comandi di riproduzione (Play/Pause) e navigazione temporale arbitraria (*seeking*) tramite cursore (*playhead*).
- **RF7 (Anteprima Reattiva WYSIWYG):** Il viewport di preview deve riprodurre in tempo reale lo stato della sequenza, applicando dinamicamente al timestamp corrente le trasformazioni geometriche, le dissolvenze e gli effetti cromatici calcolati per la clip visibile.
- **RF8 (Esportazione Asincrona del Progetto):** L'utente deve poter selezionare il file di destinazione tramite dialog di sistema e avviare il rendering finale delegato a un processo esterno FFmpeg.
- **RF9 (Monitoraggio dell'Avanzamento Rendering):** Durante l'export, il sistema deve esporre una finestra dedicata per visualizzare in tempo reale la percentuale di completamento dell'operazione e notificare eventuali errori di esecuzione.

3.2 Requisiti Non Funzionali e Vincoli di Dominio

- **RNF1 (Editing Non Distruttivo):** Le modifiche temporali, i tagli e gli effetti visivi operano esclusivamente sulla rappresentazione logica in memoria (*Timeline*), lasciando inalterati i file multimediali originali su disco.
- **RNF2 (Coerenza Computazionale Anteprima-Export):** I modelli matematici alla base delle trasformazioni visive (*EffectCalculator*) devono garantire coerenza visiva e dinamica tra l'anteprima JavaFX e il rendering generato dalla pipeline di filtri FFmpeg.
- **RNF3 (Reattività dell'Interfaccia Grafica):** Le operazioni a elevata intensità di calcolo (il rendering di FFmpeg) e le operazioni di I/O non devono bloccare l'esecuzione del thread grafico principale (*UIThread*).

4 Modello del Dominio

4.1 Entità Fondamentali e Algebraic Data Types

La rappresentazione dello stato poggia sull'impiego sinergico di tipi prodotto (*product types*, realizzati tramite *case class*) e tipi somma (*sum types*, modellati tramite *sealed trait* e gerarchie chiuse). La struttura complessiva del modulo *core.model* è sintetizzata nel diagramma delle classi concettuale riportato in Figura 1.

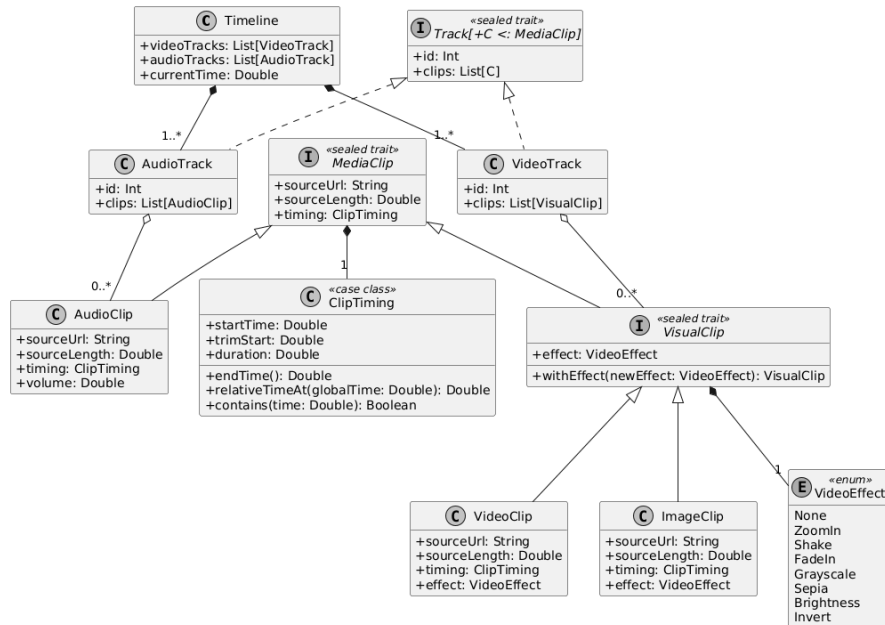


Figura 1: Diagramma concettuale del modello di dominio puro (*core.model*).

Le entità cardine del dominio sono così definite:

- **Timeline:** Rappresenta l'aggregato radice (*Root Entity*) del sistema. È un contenitore immutabile che descrive la composizione globale del progetto attraverso una collezione di tracce video (`videoTracks: List[VideoTrack]`), tracce audio (`audioTracks: List[AudioTrack]`) e l'istante di riproduzione corrente espresso in secondi (`currentTime: Double`).
- **Track[+C <: MediaClip]:** Trait covariante rispetto al tipo di clip ospitato, astraente il concetto di corsia temporale indipendente. Si specializza nelle implementazioni concrete:
 - **VideoTrack:** collezione ordinata di elementi visivi (`VisualClip`);
 - **AudioTrack:** sequenza di elementi acustici (`AudioClip`).
- **MediaClip:** Tipo algebrico somma sigillato (`sealed trait`) che incapsula le proprietà comuni a ogni risorsa multimediale inserita nel progetto:
 - **sourceUrl:** identificatore/percorso della risorsa sorgente su disco;
 - **sourceLength:** durata nativa originaria del file;
 - **timing:** metadati di posizionamento e ritaglio definiti tramite `ClipTiming`.
- **VisualClip:** Specializzazione sigillata di `MediaClip` riservata alle risorse dotate di resa visiva. Associa alla clip un effetto visivo (`effect: VideoEffect`) ed espone il metodo funzionale `withEffect` per la rigenerazione immutabile dell'istanza. Si articola in:
 - **VideoClip:** flusso video canonico a durata finita;
 - **ImageClip:** risorsa grafica fissa, concettualmente estendibile temporalmente a tempo indefinito (`sourceLength = Double.PositiveInfinity`).
- **AudioClip:** Specializzazione di `MediaClip` per flussi sonori, dotata di un fattore di guadagno scalare (`volume: Double`) e di una collezione ordinata di nodi temporali (`volumePoints: List[VolumePoint]`) per la modulazione dell'ampiezza.
- **VolumePoint:** Tipo prodotto che associa un timestamp a un livello di guadagno sonoro, modellando i punti di controllo per l'interpolazione del volume.
- **ClipTiming:** Componente deputata all'isolamento dell'algebra temporale del montaggio. Modella il posizionamento e il ritaglio attraverso tre metriche:
 - **startTime:** istante assoluto di inizio della clip sull'asse dei tempi della traccia;

- **trimStart**: offset relativo all’inizio del file sorgente da cui avviare la riproduzione;
- **duration**: estensione temporale visibile effettiva dello spezzone sulla timeline.
- **VideoEffect**: Tipo algebrico che formalizza le trasformazioni grafiche supportate, spaziando da deformazioni dinamiche e geometriche (**ZoomIn**, **Shake**) a regolazioni tonali e cromatiche (**Brightness**, **Grayscale**, **Sepia**, **Invert**) e modulazioni di trasparenza (**FadeIn**).

4.2 Invarianti di Dominio e Regole di Coerenza Temporale

La correttezza semantica del montaggio impone il rispetto di invarianti formali che devono essere preservati a fronte di qualsiasi operazione di inserimento, trimming o traslazione:

1. **Consistenza dei Parametri Temporal**: Per ogni istanza di **ClipTiming** deve valere la stretta positività della durata visibile e la non negatività degli indici temporali:

$$duration > 0 \quad \wedge \quad startTime \geq 0 \quad \wedge \quad trimStart \geq 0$$

2. **Integrità del File Sorgente (Trimming Bounds)**: Nel caso di risorse a durata finita ($sourceLength < +\infty$, es. **VideoClip** e **AudioClip**), la finestra di riproduzione delimitata dal trimming non può eccedere i confini fisici del media originale:

$$trimStart + duration \leq sourceLength$$

3. **Non Sovrapposizione sulla Traccia**: Date due clip distinte $c_1, c_2 \in T$ allocate sulla medesima traccia T , gli intervalli temporali definiti da $[startTime, endTime[$ devono essere mutuamente disgiunti:

$$\forall c_1, c_2 \in T, \quad c_1 \neq c_2 \quad (c_1.endTime \leq c_2.startTime) \vee (c_2.endTime \leq c_1.startTime)$$

4. **Mappatura delle Coordinate Temporal Relative**: Dato un tempo globale di scansione t_{global} tale per cui $c.startTime \leq t_{global} < c.endTime$, il corrispondente timestamp relativo t_{local} associato al flusso del file originario è formalizzato dalla proiezione:

$$t_{local} = (t_{global} - c.startTime) + c.trimStart$$

5 Architettura e Design del Sistema

L’architettura del sistema è strutturata separando le responsabilità in moduli a basso accoppiamento, mantenendo la logica del montaggio indipendente dall’interfaccia grafica e dagli strumenti di esportazione.

Per garantire ciò, sono stati sfruttati i seguenti pattern:

- **Model-View-Controller a Flusso Unidirezionale:** Il sistema implementa una variante disaccoppiata del pattern MVC (affine all'approccio Model-View-Presenter). Il sottosistema `core` racchiude il modello puro (`core.model`) e la logica di calcolo (`core.engine`), completamente isolati dall'infrastruttura grafica. Il componente `app.controller` funge da mediatore esclusivo: riceve le notifiche di evento da `app.view`, interpella l'engine funzionale per generare un nuovo stato e comanda il ridisegno dei componenti visivi, impedendo legami bidirezionali o dipendenze cicliche.
- **Adapter Pattern (FFmpegVideoExporter):** Isola l'interazione a basso livello con la CLI di FFmpeg e la gestione dei flussi I/O di sistema dietro un'interfaccia tipizzata in Scala, mascherando la natura imperativa del processo esterno.
- **Builder Pattern (ExportCommandBuilder):** Separa l'assemblaggio algoritmico del grafo di filtri video/audio (`-filter_complex`) e dei parametri di rendering dalla fase di esecuzione del comando.
- **Memento Funzionale per Undo/Redo (History[Timeline]):** Sfruttando l'immutabilità della `Timeline` (struttura dati persistente con condivisione strutturale della memoria), lo storico delle azioni è realizzato mantenendo due stack di snapshot, garantendo la reversibilità delle modifiche senza dover clonare i dati in memoria.

5.1 Struttura Modulare e Layer Architetture

Il sistema adotta un'architettura a livelli (*layered architecture*) in cui il flusso delle dipendenze è unidirezionale dall'esterno verso l'interno, prevenendo dipendenze cicliche. La macro-struttura è illustrata nel diagramma dei componenti di Figura 2.

I livelli architetturali principali sono così organizzati:

- **Domain Layer (`core.model`):** Collocato alla base dell'architettura, modella il dominio applicativo tramite tipi di dato algebrici (*Algebraic Data Types*) e strutture dati puramente immutabili (`Timeline`, `Track`, `MediaClip`, `ClipTiming`). È completamente privo di dipendenze verso framework grafici o processi esterni, garantendo la totale isolabilità del modello logico e consentendone la verifica attraverso test unitari deterministici.
- **Engine & Processing Layer (`core.engine`):** Rappresenta il nucleo computazionale dell'applicazione, responsabile delle trasformazioni del modello e dell'elaborazione multimediale:
 1. *Manipolazione Funzionale della Timeline (`TimelineEngine`):* incapsula le operazioni di business logic per la modifica della timeline (inserimento clip, tagli, spostamenti tra tracce, rimozioni, calcolo

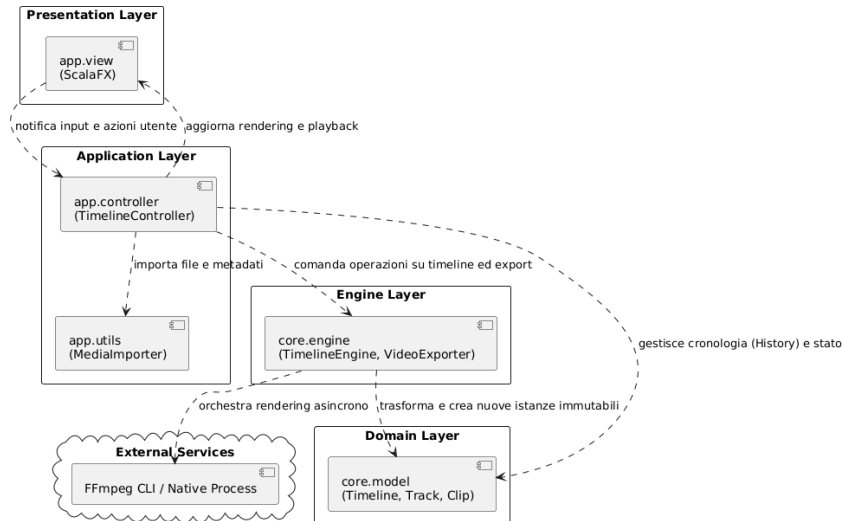


Figura 2: Architettura a layer e flusso delle dipendenze tra i sottosistemi.

dello snapping). Trattandosi di funzioni pure, ogni operazione riceve una *Timeline* e produce una nuova istanza immutabile senza effetti collaterali;

2. *Calcolo degli Effetti (EffectCalculator)*: valuta in modo deterministico le trasformazioni visive (coordinate, fattore di scala, opacità) a partire dai parametri dell'effetto e dall'istante temporale corrente;
3. *Compilazione ed Esecuzione Pipeline FFmpeg (VideoExporter)*: traduce la *Timeline* in complessi grafi di filtri (`-filter_complex`) e argomenti per FFmpeg, gestendo l'invocazione asincrona del processo nativo e il monitoraggio dell'avanzamento tramite la lettura dell'output (`stderr`).

- **Application Layer (app.controller)**: Agisce da orchestratore applicativo tramite *TimelineController*. Cattura le intenzioni utente provenienti dalla vista, delega a *TimelineEngine* il calcolo dei nuovi stati della timeline e ne gestisce la cronologia lineare (*History[Timeline]*) per abilitare le funzionalità di Undo/Redo. Coordina inoltre il ciclo di riproduzione tramite un timer asincrono e sincronizza le anteprime audio/video.
- **Presentation Layer (app.view)**: Realizzato con ScalaFX, gestisce la presentazione visiva e la raccolta degli input utente (trascinamento clip, regolazione cursori, scorciatoie da tastiera intercettate da *InputHandler*). Adotta un approccio reattivo basato sulle proprietà osservabili e proietta direttamente lo stato della timeline immutabile sugli elementi a schermo (*TimelinePanel*, *VideoPreview*, *EffectInspectorPane*).

5.2 Disaccoppiamento del Rendering: Il Ruolo di EffectCalculator

Per evitare discrepanze visive tra la visualizzazione in anteprima e l'esportazione finale, la logica di calcolo delle trasformazioni è centralizzata nel componente `EffectCalculator`:

$$f : (\text{VideoEffect}, t_{rel}, duration) \longrightarrow \text{TransformParameters}(scale, \Delta x, \Delta y, opacity)$$

Dato l'effetto e l'istante temporale relativo alla clip, il modulo restituisce una struttura dati contenente i parametri di trasformazione. Questo approccio garantisce:

1. **Coerenza Visiva (WYSIWYG):** sia la vista `ScalaFX` (tramite le proprietà grafiche in `VideoPreview`) sia il builder di comandi `FFmpeg` (tramite i filtri video `scale`, `crop` e curve di dissolvenza) condividono le stesse formule matematiche, mantenendo identici tempi e intensità nelle animazioni (es. oscillazioni o rampe di zoom);
2. **Separazione delle Responsabilità:** `VideoEffect` funge da puro descrittore dei parametri impostati dall'utente, lasciando all'engine il compito di calcolarne l'applicazione nel tempo.

5.3 Reattività dell'Interfaccia e Monitoraggio dell'Esportazione

Durante la generazione del filmato finale tramite `FFmpeg`, l'applicazione garantisce l'integrità del progetto inibendo l'interazione con l'editor principale attraverso una finestra di dialogo modale. Per evitare tuttavia il blocco del thread grafico (*UI freeze*) causato dall'attesa sincrona del processo nativo, l'esecuzione della CLI viene gestita in modalità non bloccante.

Un flusso di lettura asincrono intercetta l'output di diagnostica prodotto da `FFmpeg` su `stderr`, estraendo ad intervalli regolari il timestamp corrente di codifica. Calcolata la percentuale rispetto alla durata complessiva della sequenza, l'aggiornamento della barra di avanzamento viene ripianificato sul thread grafico tramite `Platform.runLater`. In questo modo, pur impedendo modifiche concorrenti alla timeline durante l'elaborazione, l'interfaccia mantiene attiva la barra di progresso fornendo all'utente un riscontro visivo continuo sullo stato del rendering.

6 Implementazione

6.1 Dettagli Implementativi: Motore di Editing ed Elaborazione Video

Il package `core.engine` costituisce il nucleo computazionale dell'applicazione.

6.1.1 Manipolazione della Timeline (TimelineEngine)

Il modulo `TimelineEngine` costituisce il punto di ingresso per qualsiasi alterazione dello stato di editing. Ogni operazione, sia essa di inserimento, rimozione, trimming, scissione o traslazione, riceve l'istanza corrente di `Timeline` e ne restituisce una nuova copia, preservando l'integrità referenziale e lo storico delle modifiche.

Tra le procedure cardine del motore abbiamo:

- **Politiche di Inserimento (`resolveInsertAndShift`):** Quando una clip viene inserita lungo una coordinata temporale già occupata, la funzione divide automaticamente lo spezzone preesistente in due segmenti distinti (preservando coerentemente offset sorgente e durate) e trasla in avanti di una quantità pari alla durata della nuova clip tutti gli elementi successivi, sfruttando pattern matching e combinatori su collezioni.
- **Compattazione Sequenziale (`snapClipsGeneric`):** Implementa una routine funzionale basata su `foldLeft` per riordinare cronologicamente e riposizionare sequenzialmente le clip lungo l'asse temporale, eliminando interruzioni e vuoti tra elementi contigui:

```
1 private def snapClipsGeneric[C <: MediaClip](clips: List[C]):  
2   List[C] =  
3   clips.sortBy(_.startTime).foldLeft(List.empty[C]): (  
4     accumulated, currentClip) =>  
5     val nextStartTime = accumulated.lastOption.map(_.endTime).  
6     getOrElse(0.0)  
     accumulated :+ currentClip.withTiming(  
       currentClip.timing.copy(startTime = nextStartTime)  
     )
```

Listing 1: Allineamento temporale funzionale tramite `foldLeft`

6.1.2 Costruzione Dichiarativa della Pipeline FFmpeg (ExportCommandBuilder)

La traduzione della struttura logica del montaggio in istruzioni eseguibili per la CLI di FFmpeg è delegata a `ExportCommandBuilder`. Il componente converte la composizione di tracce e clip in un singolo grafo aciclico diretto di elaborazione filtrata attraverso l'opzione `-filter_complex`:

1. **Normalizzazione e Deduplicazione Input:** Gli URL dei file sorgente associati alle clip vengono deduplicati per ottimizzare l'I/O, assegnando a ciascuna risorsa su disco un indice numerico di input progressivo (`-i file`).
2. **Generazione del Canvas Base:** Viene istanziato un flusso video sintetico neutro (`color=c=black`) configurato con i parametri di esportazione (`width, height, fps`) ed esteso per l'intera durata della timeline.

3. **Composizione a Livelli (Overlay Chains):** Per ciascuna clip visiva (immagine statica con loop temporale o spezzone video) viene generata una catena di filtri dedicata comprendente ritaglio, adattamento d'aspetto (scale e pad) e rendering degli effetti. I nodi risultanti vengono sovrapposti sequenzialmente sul canvas principale tramite filtri overlay regolati dall'espressione di abilitazione temporale `enable='between(t, start, end)'`.
4. **Mixaggio Audio Multitraccia:** I flussi sonori vengono sincronizzati mediante inserimento del ritardo di avvio tramite il filtro `adelay`, regolati in ampiezza (volume) e convogliati in un unico nodo di missaggio stereo mediante `amix`.

```

1 // 1. Normalizzazione e Deduplicazione Input
2 val allUniqueFiles = (videoClipsWithTrack.map(_._2.sourceUrl
3 ) ++ audioClipsWithTrack.map(_._2.sourceUrl))
4   .distinct.map(extractFilePath)
5 val fileIndexMap = allUniqueFiles.zipWithIndex.toMap
6 val inputArgs    = allUniqueFiles.flatMap(file => Seq("-i",
7   file))
8
9 // 2. Generazione del Canvas Base
10 val baseVideoFilter = s"color=c=black:s=${settings.width}x${
11 settings.height}:r=${settings.fps}:d=${fmt(totalDuration)}[
12 bg0]"
13
14 // 3. Composizione a Livelli: catene di filtri e overlay
15 sequenziali
16 val videoFilterParts = videoClipsWithTrack.zipWithIndex.map
17 { case (_, clip), idx =>
18   val fileIdx = fileIndexMap(extractFilePath(clip.sourceUrl))
19   val clipChain = clip match
20     case _: ImageClip => Seq(Some("loop=-1:1:0"),
21   formatEffect(clip.effect, clip.duration), Some(scaleFilter),
22   /* ... */).flatten.mkString(",")
23     case _: VideoClip => Seq(Some(s"trim=start=${fmt(clip.
24   trimStart)}:duration=${fmt(clip.duration)}"), /* ... */).
25   flatten.mkString(",")
26   s"[$fileIdx:v]$clipChain[v$idx]"
27   }
28
29 val overlayChains = videoClipsWithTrack.indices.map { idx =>
30   val prevBg = if idx == 0 then "[bg0]" else s"[bg$idx]"
31   val outBg  = if idx == videoClipsWithTrack.size - 1 then "
32 [vout]" else s"[bg${idx + 1}]"
33   val (start, end) = (fmt(videoClipsWithTrack(idx)._2.
34   startTime), fmt(videoClipsWithTrack(idx)._2.endTime))
35   s"$prevBg[v$idx]overlay=x=0:y=0:enable='between(t,$start,
36   $end)',$outBg"
37   }
38
39

```

```

26 // 4. Missaggio Audio Multitraccia: ritardo (adelay) e
    missaggio (amix)
27 val audioFilterParts = audioClipsWithTrack.zipWithIndex.map
    { case ((_, clip), idx) =>
28     val fileIdx = fileIndexMap(extractFilePath(clip.sourceUri)
    )
29     val delayMs = (clip.startTime * 1000).toLong
30     val delayFilter = if delayMs > 0 then s",adelay=$delayMs|
    $delayMs" else ""
31     s"[$fileIdx:a]atrim=start=${fmt(clip.trimStart)}:duration=
    ${fmt(clip.duration)},volume=${fmt(clip.volume)}$delayFilter[
    a$idx]"
32     }
33     val mixFilter = s"${inputsMerged}amix=inputs=${
    audioClipsWithTrack.size}:duration=longest:dropout_transition
    =0:normalize=0[aout]"

```

Listing 2: Costruzione dichiarativa della pipeline in `ExportCommandBuilder`

6.1.3 Esecuzione Asincrona e Monitoraggio dell'Avanzamento (VideoExporter)

Il controllo e la sorveglianza del processo di rendering sono demandati a `FFmpegVideoExporter`, la cui invocazione restituisce un tipo monadico `Try[File]` a presidio della gestione sicura dei fallimenti a livello di sistema operativo.

Per scongiurare il blocco dell'interfaccia utente durante la codifica e fornire un feedback continuo, l'esecuzione combina `ProcessBuilder` con lo streaming diagnostico:

- L'eseguibile viene invocato con i flag `-nostats` e `-progress pipe:1`, indirizzando lo stato computazionale su standard output.
- La direttiva `redirectErrorStream(true)` unifica i canali di diagnostica su un unico `BufferedReader`.
- La routine `parseProgress` analizza in tempo reale le righe emesse tramite espressioni regolari (`Regex` sul campo `out_time`), estrapola il timestamp elaborato e calcola la frazione percentuale di avanzamento notificata alla callback `onProgress`:

```

1 private val outTimePattern =
2   """(?:out_time|time)=(-?\d{2}):(\d{2}):(\d{2})(?:\.\d+)?""" .r
3
4 outTimePattern.findFirstMatchIn(line).foreach: m =>
5   val hours = m.group(1).toDouble
6   val minutes = m.group(2).toDouble
7   val seconds = m.group(3).toDouble
8   val currentSeconds = (hours * 3600.0) + (minutes * 60.0) +
    seconds
9   val fraction = Math.min(1.0, Math.max(0.0, currentSeconds /
    totalDuration))

```

```
10 onProgress(fraction)
```

Listing 3: Parsing con Regex del timestamp di avanzamento di FFmpeg

6.1.4 Logica Deterministica degli Effetti Visivi tramite Typeclass

A completamento dell'engine, la formulazione delle trasformazioni grafiche applicabili alle clip è centralizzata in `EffectCalculator`. Tale modulo assicura parità di comportamento (WYSIWYG) tra l'anteprima JavaFX e il rendering finale generato da FFmpeg.

L'implementazione sfrutta il pattern **Typeclass**, valorizzando i costrutti nativi di **Scala 3** (trait, clause contestuali `given` e parametri `using`):

- Il trait `EffectCalculator[E]` definisce la computazione della trasformazione geometrica pura (`EffectTransform`) a partire dall'effetto generico *E*, dal tempo relativo e dalla durata della clip.
- Vengono dichiarate istanze contestuali (`given`) specifiche per ogni variante supportata (`ZoomIn`, `Shake`, `FadeIn`).
- Un *extension method* arricchisce i sottotipi di effetto, abilitando la sintassi `effect.computeTransform(relativeTime, duration)`.

```
1 trait EffectCalculator[E]:  
2   def compute(effect: E, relativeTime: Double, clipDuration:  
3     Double): EffectTransform  
4  
5 object EffectCalculator:  
6   extension [E](effect: E)(using calc: EffectCalculator[E])  
7     def computeTransform(relativeTime: Double, clipDuration:  
8       Double): EffectTransform =  
9       calc.compute(effect, relativeTime, clipDuration)  
10  
11 given EffectCalculator[VideoEffect.Shake] with  
12   def compute(eff: VideoEffect.Shake, relativeTime: Double,  
13     clipDuration: Double): EffectTransform =  
14     val dx = eff.intensity * Math.sin(2 * Math.PI * eff.freq *  
15       relativeTime)  
16     val dy = eff.intensity * Math.cos(2 * Math.PI * (eff.freq  
17       * 1.3) * relativeTime)  
18     EffectTransform(translateX = dx, translateY = dy)
```

Listing 4: Definizione della Typeclass e istanza Shake in Scala 3

Le equazioni trigonometriche formalizzate nella funzione `compute` vengono tradotte direttamente da `ExportCommandBuilder` nelle corrispondenti espressioni analitiche dei filtri FFmpeg (`sin(wx*t)` e `cos(wy*t)`), assicurando che oscillazioni, zoom e rampe di dissolvenza coincidano fedelmente tra l'anteprima a schermo e l'output codificato su file.

6.2 Dettagli Implementativi: Interfaccia Utente e Coordinamento Reattivo

Il livello di presentazione (package `app.view`) è sviluppato mediante il framework **ScalaFX**. L'interfaccia è concepita secondo il paradigma reattivo: i componenti grafici non manipolano direttamente lo stato applicativo, ma espongono proprietà osservabili (`ObjectProperty`) e notificano le interazioni utente al Controller mediante funzioni di callback di ordine superiore.

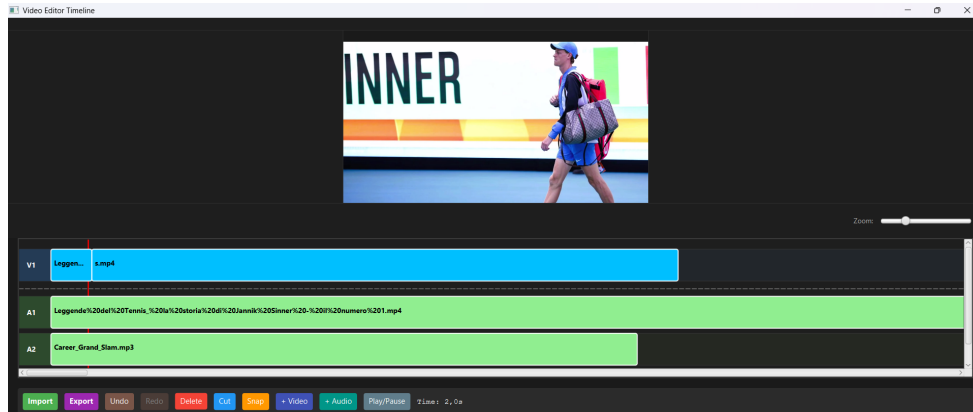


Figura 3: Interfaccia grafica principale dell'editor (`app.view`)

6.2.1 Composizione Visuale della Timeline e Manipolazione Diretta

Il componente centrale dell'interfaccia è `TimelinePanel`, un contenitore personalizzato derivato da `Pane` che implementa il rendering procedurale e l'interazione diretta con le tracce:

- **Proiezione Spazio-Temporale:** L'asse temporale è convertito in coordinate pixel tramite il parametro dinamico `pixelsPerSecond`, regolabile mediante slider di zoom. Il tempo assoluto t viene linearmente mappato sulla coordinata orizzontale:

$$X(t) = trackLabelWidth + (t \cdot pixelsPerSecond)$$

La barra di riproduzione (*playhead*) e gli elementi multimediali vengono posizionati sul canvas in base a questa trasformazione affine.

- **Drag-and-Drop Multitraccia:** Ciascuna clip è incapsulata in un `Rectangle` associato a listener di eventi del mouse. Durante il trascinamento orizzontale e verticale, la vista aggiorna le traslazioni locali (`translateX`, `translateY`). Al rilascio (`onMouseReleased`), la posizione cartesiana viene convertita nelle coordinate di dominio (nuovo `startTime` e traccia di destinazione calcolata tramite la minima distanza euclidea tra i centri delle tracce), inoltrando la richiesta al callback puro `onMove`:

```

1  val finalX = Math.max(trackLabelWidth, initialX + clipRectangle.
    translateX.value)
2  val newStartTime = (finalX - trackLabelWidth) / pixelsPerSecond
3  val currentLocalCenterY = yPos + clipRectangle.translateY.value
    + (trackHeight / 2.0)
4
5  val targetTrackId = tracksMeta
6    .minByOption(t => Math.abs(currentLocalCenterY - (t._2 +
    trackHeight / 2.0)))
7    .map(_._1)
8    .getOrElse(sourceTrackId)
9
10 onMove(mediaClip, targetTrackId, newStartTime)

```

Listing 5: Risoluzione geometrica del target track durante il drag-and-drop

6.2.2 Sincronizzazione della Preview e Rendering degli Effetti

Il rendering in tempo reale dello stato del progetto è delegato al componente `VideoPreview`. Per minimizzare l'impronta computazionale ed evitare allocazioni ridondanti, la classe adotta un meccanismo a doppio visualizzatore:

1. **Gestione Ibrida Media/Immagini:** Differenzia tramite estensione del file le immagini fisse (renderizzate tramite `ImageView`) dai flussi video (affidati a `MediaView`). In presenza di immagini statiche, l'eventuale istanza preesistente di `MediaPlayer` viene immediatamente arrestata e deallocata (`dispose`) per rilasciare i descrittori di file nativi.
2. **Integrazione con `EffectCalculator`:** A ogni aggiornamento temporale, la vista interroga la typeclass dell'engine per ricavare la tupla geometrica `EffectTransform`. I valori calcolati vengono applicati ai nodi dello Scene Graph:

```

1  val transform = EffectCalculator.computeTransform(effect,
    relativeTime, duration)
2  node.scaleX = transform.scale
3  node.scaleY = transform.scale
4  node.translateX = transform.translateX
5  node.translateY = transform.translateY
6  node.opacity = transform.opacity
7
8  effect match
9    case VideoEffect.Grayscale =>
10      val ca = new ColorAdjust(); ca.setSaturation(-1.0); node.
        effect = ca
11    case VideoEffect.Sepia =>
12      val sepia = new SepiaTone(); sepia.setLevel(0.8); node.
        effect = sepia
13    case VideoEffect.Brightness(level) =>
14      val ca = new ColorAdjust(); ca.setBrightness(level); node.
        effect = ca

```

```

15 case _ =>
16     node.effect = null

```

Listing 6: Applicazione reattiva dei parametri di trasformazione ed effetti cromatici

Questa suddivisione garantisce che la vista si limiti ad applicare proprietà grafiche, delegando interamente la formulazione delle curve matematiche alla logica di dominio dell'engine.

6.2.3 Mixer Audio Multitraccia e Gestione dello Stato Concorrente

La riproduzione concorrente delle tracce sonore è gestita dalla classe `AudioPlayer`. A differenza del flusso video principale (dove il viewport mostra un solo fotogramma alla volta), sulla timeline possono coesistere molteplici flussi audio attivi nel medesimo istante t .

Per prevenire corse critiche (*race conditions*) tra il thread grafico di JavaFX e le invocazioni di aggiornamento temporale, il componente mantiene una mappa immutabile protetta da un riferimento atomico (`AtomicReference[Map[String, MediaPlayer]]`):

- **Riconciliazione Funzionale:** A ogni invocazione di `update`, la collezione dei player correnti viene partizionata (`partition`) confrontando gli URL attivi: le sorgenti non più presenti nel timestamp corrente vengono arrestate e rilasciate mediante `dispose()`.
- **Sincronizzazione e Compensazione di Drift:** Per i media ancora attivi, il player calcola la discrepanza tra l'istante effettivo di riproduzione e il tempo teorico della timeline. Se la differenza temporale eccede una soglia di tolleranza prefissata (0.3 s), viene forzata un'operazione correttiva di riallineamento (`seek`).

6.2.4 Ispezione Contestuale dei Parametri (`EffectInspectorPane`)

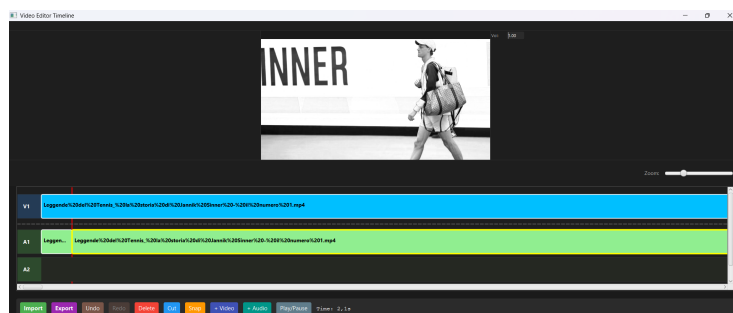


Figura 4: Pannello `EffectInspectorPane` aperto per la gestione dell'audio selezionato

Per consentire la modifica fine dei parametri associati a ciascun elemento, la classe `EffectInspectorPane` realizza un pannello di ispezione reattivo che si adatta dinamicamente alla selezione corrente:

- Tramite pattern matching sull'ADT `MediaClip`, il pannello identifica la tipologia di elemento selezionato: per le istanze di `VisualClip` espone i campi di regolazione dell'effetto associato (es. fattore di ingrandimento per `ZoomIn`, frequenza e intensità per `Shake`, durata per `FadeIn`), mentre per le istanze di `AudioClip` renderizza i controlli di guadagno (volume).
- Ogni modifica immessa nei campi di testo (`TextField`) viene validata, limitata all'interno dell'intervallo operativo ammissibile (*clamping*) e propagata verso il controller attraverso callback parametriche, aggiornando in tempo reale sia la preview sia il modello logico.

6.3 Dettagli Implementativi: Il controller

6.3.1 Gestione Atomica dello Stato e Cronologia (Single Source of Truth)

Per evitare inconsistenze tra thread concorrenti e preservare l'approccio funzionale, l'intero stato dinamico dell'applicazione è incapsulato nella struttura immutabile `ControllerState`:

```
1 private case class ControllerState(  
2   history: History[Timeline],  
3   currentTime: Double = 0.0,  
4   playerState: PlayerState = Paused,  
5   lastFrameTimeNanos: Long = 0L  
6 ):  
7   def timeline: Timeline = history.current
```

Listing 7: Struttura dati atomica dello stato del Controller

L'istanza corrente è protetta da un `AtomicReference[ControllerState]`. Ogni modifica allo stato del progetto (inserimento, rimozione, taglio o traslazione di tracce e clip) transita da una pipeline di aggiornamento centralizzata:

1. La vista notifica un'azione utente invocando la corrispondente callback del controller.
2. Il controller delega l'elaborazione a `TimelineEngine`, ottenendo una nuova istanza immutabile di `Timeline`.
3. Il metodo unificato `updateTimelineState` inserisce la nuova timeline nello stack `History[Timeline]`, imposta il nuovo stato atomico e orchestra il riallineamento di `view.render`, dei controlli Undo/Redo e dei player multimediali.

6.3.2 Playback Loop e Sincronizzazione Temporale Differenziale

La progressione temporale durante la riproduzione è governata da un `AnimationTimer` di JavaFX (`masterTimer`), che calcola il tempo trascorso a livello di nanosecondi:

$$\Delta t = \frac{now - lastFrameTimeNanos}{10^9}$$

Il controller aggiorna il timestamp corrente moltiplicando Δt per la velocità di riproduzione nominale. Per evitare ricaricamenti ridondanti delle risorse multimediali, la sincronizzazione attiva (`syncMediaPlayback`) viene innescata esclusivamente al verificarsi di precise condizioni di variazione:

- Transizione di confine tra clip video o sonore adiacenti (`prevVideo != currVideo` o `prevAudios != currAudios`);
- Raggiungimento del termine della timeline con arresto automatico;
- Presenza di effetti grafici parametrici continui nel tempo (`ZoomIn`, `Shake`, `FadeIn`), i quali richiedono una ricalibrazione fotogramma per fotogramma tramite `EffectCalculator`.

6.3.3 Elaborazione Asincrona dell'Esportazione con Future

Le operazioni di rendering e codifica video, data la loro natura intensiva per la CPU, non possono essere eseguite sul thread primario di JavaFX senza causare il congelamento dell'interfaccia utente. Il controller isola quindi l'invocazione di `VideoExporter` all'interno di un'elaborazione asincrona non bloccante:

```
1 Future {
2     exporter.exportVideo(
3         timeline = current.timeline,
4         settings = settings,
5         onProgress = { fraction =>
6             Platform.runLater {
7                 progressBar.progress = fraction
8                 val percent = Math.min(100, (fraction * 100).toInt)
9                 progressLabel.text = s"Esportazione in corso: $percent%"
10            }
11        }
12    )
13 }.flatMap(res => Future.fromTry(res)).onComplete {
14     case Success(file) =>
15         Platform.runLater {
16             dialogStage.close()
17             // Notifica di successo su UI Thread
18         }
19     case Failure(exception) =>
20         Platform.runLater {
21             dialogStage.close()
22             // Notifica di errore su UI Thread
23         }
24 }
```

24 }

Listing 8: Orchestrazione asincrona del rendering FFmpeg

La callback `onProgress` sfrutta `Platform.runLater` per inoltrare i campionamenti parziali estratti dal parser CLI direttamente ai controlli visuali della finestra modale (`ProgressBar`), preservando la totale reattività dell'applicazione durante l'elaborazione.

7 Strategia di Testing e Validazione

La validazione della correttezza del sistema è stata condotta mediante una suite di test automatizzati sviluppata con il framework **ScalaTest**, adottando lo stile dichiarativo `AnyFunSuite` con il supporto del mixin `Matchers`.

7.1 Verifica degli Invarianti di Manipolazione

Di seguito vengono analizzati i test più significativi implementati nella classe `TimelineEngineTest`.

7.1.1 Preservazione dell'Immutabilità e Robustezza agli Indici Invalidi

Il test di immutabilità verifica che l'aggiunta di una clip non muti la traccia originaria, salvaguardando l'assenza di effetti collaterali:

```
1 test("Ensure immutability by keeping the starting timeline  
   unchanged") {  
2   val _ = TimelineEngine.addVideoClip(initialTimeline, trackId =  
     1, clip = sampleClip)  
3  
4   initialTimeline.videoTracks.head.clips shouldBe empty  
5 }
```

Listing 9: Verifica dell'immutabilità della Timeline di partenza

Specularmente, la rimozione con un indice inesistente (`clipIndex = 999`) dimostra la stabilità del motore, che restituisce l'istanza originale senza sollevare eccezioni impreviste:

```
1 test("Return the exact same timeline if the clip index to remove  
   does not exist") {  
2   val updatedTimeline = TimelineEngine.removeVideoClip(  
     initialTimeline, trackId = 1, clipIndex = 999)  
3  
4   updatedTimeline shouldBe initialTimeline  
5 }
```

Listing 10: Gestione sicura di indici non validi

7.1.2 Taglio Deterministico e Sincronizzazione Multitraccia

L'operazione di taglio (`cutVideoClip`) valida sperimentalmente l'invariante di consistenza temporale: la somma delle durate delle due nuove clip equivale a quella del segmento sorgente, e l'offset `trimStart` del segmento destro corrisponde esattamente al punto di taglio:

```
1 test("Cut a video clip into two distinct sub-clips at a relative
   time") {
2   val clipToCut = VideoClip(
3     sourceUrl = "video1.mp4", startTime = 10.0, trimStart = 0.0,
4     duration = 10.0, sourceLength = 20.0, effect = VideoEffect.
       None
5   )
6   val timelineWithClip = TimelineEngine.addVideoClip(
       initialTimeline, trackId = 1, clip = clipToCut)
7
8   val updatedTimeline = TimelineEngine.cutVideoClip(
       timelineWithClip, trackId = 1, clipIndex = 0, relativeCutTime
       = 4.0)
9   val finalClips = updatedTimeline.videoTracks.head.clips
10
11   finalClips should have size 2
12   val leftClip = finalClips(0)
13   leftClip.startTime shouldBe 10.0; leftClip.duration shouldBe
       4.0; leftClip.trimStart shouldBe 0.0
14
15   val rightClip = finalClips(1)
16   rightClip.startTime shouldBe 14.0; rightClip.duration shouldBe
       6.0; rightClip.trimStart shouldBe 4.0
17 }
```

Listing 11: Test di scissione temporale di una clip video

La medesima logica viene testata sull'azione coordinata `cutVideoAndAudio`, verificando che il taglio simultaneo mantenga la traccia sonora originaria sincronizzata alla corrispondente traccia video.

7.1.3 Inserimento con Ripartizione e Shift Temporale

Il collaudo dell'inserimento (INSERT mode) comprova la capacità dell'algoritmo di scindere una clip preesistente all'arrivo di un secondo elemento multimediale, traslando temporalmente in avanti la porzione rimanente:

```
1 test("addVideoClip using INSERT mode should split the existing
   clip and shift the subsequent ones") {
2   val initialTrack = VideoTrack(id = 1, clips = Nil)
3   val emptyTimeline = Timeline(videoTracks = List(initialTrack),
       audioTracks = Nil)
4   val timelineWithFirstClip = TimelineEngine.addVideoClip(
       emptyTimeline, 1, sampleClip)
5   val resultingTimeline = TimelineEngine.addVideoClip(
       timelineWithFirstClip, 1, secondSampleClip)
```

```

6  val resultingClips = resultingTimeline.videoTracks.find(_.id =
    = 1).get.clips
7
8  assert(resultingClips.size == 3)
9  val sortedClips = resultingClips.sortBy(_.startTime)
10
11  // Prima porzione della clip originaria
12  assert(sortedClips(0).startTime == 0.0 && sortedClips(0).
    duration == 5.0)
13  // Clip inserita
14  assert(sortedClips(1).startTime == 5.0 && sortedClips(1).
    duration == 5.0)
15  // Seconda porzione traslata in avanti
16  assert(sortedClips(2).startTime == 10.0 && sortedClips(2).
    duration == 5.0)
17 }

```

Listing 12: Verifica dell'inserimento con suddivisione e scorrimento automatico

7.1.4 Compattazione Magnetica (Snapping) e Gestione del Disordine

La suite dimostra l'efficacia delle funzioni `snapClipsTogether`, `snapAudioClips` e `snapAllTracks`. Il test seguente verifica che, a fronte di una lista di clip fornite in ordine arbitrario, il sistema effettui preliminarmente un ordinamento cronologico prima di ricollocare i timestamp a ridosso, eliminando i vuoti senza provocare sovrapposizioni:

```

1  test("Snap must sort clips by time before snapping to prevent
    gaps when clips are out of list order") {
2    val disorderedAudio1 = AudioClip("a1.mp3", startTime = 15.0,
    trimStart = 0.0, duration = 5.0, sourceLength = 20.0,
    volumePoints = Nil)
3    val disorderedAudio2 = AudioClip("a2.mp3", startTime = 2.0,
    trimStart = 0.0, duration = 4.0, sourceLength = 20.0,
    volumePoints = Nil)
4
5    val track = AudioTrack(id = 1, clips = List(disorderedAudio1,
    disorderedAudio2))
6    val timeline = Timeline(videoTracks = Nil, audioTracks = List(
    track))
7
8    val updatedTimeline = TimelineEngine.snapAudioClips(timeline,
    trackId = 1)
9    val snappedClips = updatedTimeline.audioTracks.head.clips
10
11    snappedClips(0).startTime shouldBe 0.0
12    snappedClips(0).sourceUrl shouldBe "a2.mp3"
13    snappedClips(1).startTime shouldBe 4.0
14    snappedClips(1).sourceUrl shouldBe "a1.mp3"
15 }

```

Listing 13: Verifica del riordinamento e compattazione delle clip

7.1.5 Polimorfismo delle Clip Visive (ImageClip)

Viene infine verificata l'astrazione polimorfica delle corsie video: `VideoTrack` è in grado di alloggiare sia flussi `VideoClip` sia istanze di `ImageClip`. Il test sul taglio di un'immagine statica (`ImageClip`) accerta che entrambi i segmenti risultanti mantengano intatto il sottotipo concreto, convalidando la correttezza del pattern matching e della funzione di copia del modello.

7.1.6 Ulteriori test

Oltre alla suite centrale sul motore di editing (`TimelineEngineTest`), la correttezza del sistema è validata da test mirati sugli altri componenti del package `core.engine`:

- **EffectCalculatorTest:** verifica la correttezza delle funzioni di trasformazione geometrica e visiva (calcolo di coordinate, fattori di scala e curve di opacità) in funzione del tempo di riproduzione.
- **ExportCommandBuilderTest:** convalida la traduzione dello stato della timeline nei parametri a riga di comando e nei grafi di filtri (`-filter_complex`) destinati a FFmpeg.
- **HistoryTest:** verifica la gestione della cronologia degli stati, accertando il corretto funzionamento delle operazioni di annullamento e ripristino (*undo/redo*) e l'integrità del modello immutabile.
- **TimelineEffectTest:** accerta la corretta associazione, modifica e rimozione degli effetti visivi e sonori applicati alle singole clip all'interno delle tracce.
- **TimelinePlaybackTest:** controlla la logica di avanzamento temporale e la corretta individuazione dei fotogrammi e delle clip attive durante la simulazione della riproduzione.
- **VideoExporterTest:** valida il flusso complessivo di esportazione e la gestione del ciclo di vita del processo esterno, verificando la corretta terminazione e l'intercettazione di eventuali errori.

8 Conclusioni e Sviluppi Futuri

Il presente progetto si colloca all'interno di un'iniziativa di sviluppo più ampia e modulare, concepita per coniugare i paradigmi della programmazione funzionale nel settore multimediale con le potenzialità dei modelli generativi e dell'intelligenza artificiale. Conformemente a quanto concordato nel piano formativo, l'architettura complessiva è articolata in due blocchi autonomi ma sinergici:

- il **Core Multimediale e Motore NLE**, oggetto esclusivo di questo elaborato e affrontato individualmente;
- il **Modulo di Automazione Intelligente**, estensione successiva progettata per operare come orchestratore ad alto livello sopra le primitive di montaggio qui realizzate.

Il lavoro svolto ha portato a compimento gli obiettivi prefissati per questo primo stadio, consegnando un'infrastruttura desktop in **Scala 3**, **Scala-FX** e **FFmpeg** solida, priva di effetti collaterali e pronta all'interoperabilità automatizzata.

8.1 Sintesi del Lavoro Svolto e Raggiungimento degli Obiettivi

Il motore NLE sviluppato soddisfa integralmente i requisiti architetturali e operativi stabiliti:

- **Modellazione Funzionale Deterministica:** L'uso di (*ADT*) e collezioni immutabili nel modulo `core.model` garantisce che ogni stato della timeline sia una fotografia pura e verificabile. Questa scelta semplifica l'annullamento/ripristino (*Undo/Redo*) e costituisce un prerequisito essenziale per consentire a un futuro agente intelligente di manipolare la sequenza temporale.
- **Motore di Editing Robusto (TimelineEngine):** Sono state implementate con successo le primitive di manipolazione multitraccia.
- **Calcolo Geometrico tramite Typeclass (EffectCalculator):** La formulazione delle trasformazioni grafiche (scaling, fade, shake) mediante typeclass in Scala 3 assicura la coerenza visiva WYSIWYG tra la preview a schermo a 60 FPS e il rendering finale via FFmpeg.
- **Integrazione e Monitoraggio Asincrono di FFmpeg:** La pipeline complessa di filtraggio (`-filter_complex`) traduce l'intera composizione in un comando nativo, monitorandone l'avanzamento su standard output in modo asincrono senza degradare la reattività dell'interfaccia grafica.

8.2 Autovalutazione e Criticità

L'analisi retrospettiva dell'architettura evidenzia i confini operativi e i compromessi tecnici emersi durante la realizzazione:

- **Trade-off tra Correttezza Formale e Micro-ottimizzazione:** Lo sforzo progettuale primario è stato deliberatamente indirizzato alla robustezza della struttura dati e alla verifica formale degli invarianti logici del dominio (`core.model`). Di conseguenza, non sono state implementate tecniche spinte di ottimizzazione delle prestazioni a basso livello, tipiche dei programmi di montaggio video moderni.

- **Limitazioni Architetture Attuali:** Sotto il profilo operativo, l'implementazione corrente presenta specifici vincoli funzionali:
 - *Singolarità dell'Effetto Visivo:* Ogni istanza di `VisualClip` supporta al momento un singolo descrittore `VideoEffect`, escludendo la composizione in cascata di più filtri simultanei sulla medesima clip (es. combinazione concorrente di `Shake` e `Grayscale`);
 - *Dipendenza dal Motore di Playback Desktop:* La riproduzione in anteprima è vincolata ai codec multimediali supportati nativamente dal motore JavaFX.

Escludendo queste consapevoli limitazioni, il software soddisfa tutti i requisiti funzionali e non decisi in partenza, in modo solido.

8.3 Sviluppi Futuri

Le prospettive evolutive del sistema sono state ripartite secondo due orizzonti temporali ben distinti: le estensioni prioritarie, concepite specificamente come ponte architetture verso il modulo di intelligenza artificiale previsto per il modulo Advanced, e i perfezionamenti tecnici a lungo termine, orientati all'ottimizzazione e alla persistenza.

8.3.1 Obiettivi Prioritari per il Modulo Advanced Software Engineering and Development (Integrazione con Agenti IA)

Il nucleo applicativo sviluppato è stato deliberatamente progettato per fungere da backend deterministico per un sistema ad agenti guidato da modelli linguistici (LLM). In tale ottica, le estensioni indispensabili comprendono:

- **Interazione tramite Prompt e Automazione a Doppio Livello:** Esposizione delle API funzionali di `TimelineEngine` verso un'interfaccia a linguaggio naturale. Il sistema ad agenti dovrà consentire sia l'automazione reattiva su richiesta dell'utente (es. *ji taglia la clip selezionata al secondo 5 e applica un effetto di dissolvenza*), sia la generazione autonoma *ex-novo* di interi progetti video partendo da un semplice prompt tematico.
- **Generazione di Immagini e Dinamismo Visivo a Basso Consumo di Token:** La scelta architetture di modellare nativamente le immagini statiche (`ImageClip`) e dotarle di trasformazioni matematiche dinamiche (`ZoomIn`, `Shake`, `FadeIn`) è stata orientata alla sostenibilità computazionale del modulo generativo. La sintesi diretta di flussi video da parte dei modelli fondazionali presenta infatti costi di generazione proibitivi e un consumo massivo di token. L'architettura consente all'agente di:
 1. generare asset grafici discreti a basso consumo di token (immagini statiche tramite prompt testuale);

2. orchestrare la durata temporale dell'asset sulla traccia visiva;
3. applicare programmaticamente gli effetti di movimento predefiniti nel modello di dominio, conferendo dinamismo cinematografico a contenuti altrimenti statici con impatto computazionale ridotto.

8.3.2 Evoluzioni a Lungo Termine ed Efficienza di Sistema

In una fase successiva, l'applicativo potrà essere arricchito con caratteristiche destinate a consolidare la fruibilità e l'efficienza complessiva dell'ambiente di lavoro:

- **Persistenza e Serializzazione dei Progetti (JSON):** Definizione di uno schema di interscambio serializzato in formato JSON per l'intero aggregato **Timeline**. Ciò consentirà il salvataggio su disco e il ripristino delle sessioni di editing, agevolando al contempo il passaggio strutturato di istruzioni tra l'utente, l'interfaccia e l'agente esterno.
- **Ottimizzazione delle Prestazioni e Gestione delle Risorse:** Raffinamento degli algoritmi interni per ridurre il carico sulla CPU e sulla memoria di sistema, introducendo meccanismi di caching mirato per i frame decodificati e strategie di rilascio conservativo delle risorse multimediali allocate dal runtime.
- **Stabilità Operativa e Resilienza dell'I/O:** Potenziamento del monitoraggio asincrono dei processi di sistema legati alla CLI di FFmpeg e consolidamento della robustezza dell'interfaccia grafica a fronte di file sorgente corrotti o formati multimediali atipici.